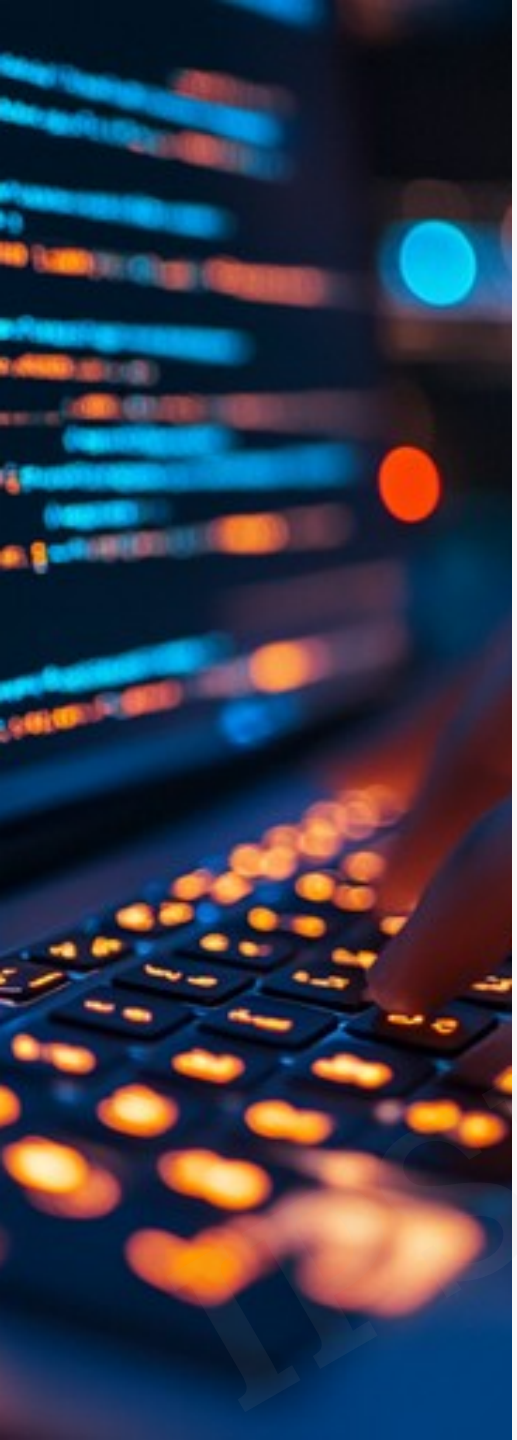


第五章、實際實例：Arm 處理器 - 01

李建緯
雲科大資工系 智慧感知與系統整合實驗室
IPSI Lab, Yuntech-CSIE

IPSI Lab, Yuntech-CSIE

Arm 簡介

Arm

- **Arm 架構**：以精簡指令集（RISC，Reduced Instruction Set Computer）理念發展的處理器架構，兼顧效能與能源效率；Arm 的歷史名稱包括 Advanced RISC Machines。
- **應用範疇**：行動裝置、嵌入式系統、IoT（Internet of Things，物聯網）、電腦與伺服器。
- **先分清層次**：Armv7 / Armv8 是架構版本；A32（傳統 Arm 32-bit 指令集） / T32（Thumb 指令集） / A64（AArch64 指令集）是指令集；Cortex、ARM926EJ-S 等是核心或其系列。

IPSI Lab, Yuntech-CSIE

Arm 的暫存器架構

Arm 的通用暫存器（General-Purpose Registers）

- **R0 - R12**：共 **13 個**可供一般運算使用的暫存器（以下以 AArch32 說明）。
  - **R0 - R3**：依 AAPCS32（Procedure Call Standard for the Arm Architecture，32-bit 版本）傳遞前四個適合核心暫存器傳遞的參數、部分返回值及暫時資料；**跨函式呼叫不保證維持舊值**。
  - **R4 - R8、R10 - R11**：被呼叫函式若使用，須在返回前恢復原值；**R9** 由平台 ABI（Application Binary Interface，應用程式二進位介面）規範用途。
  - **R12（IP，Intra-Procedure-call scratch register）**：暫時用途暫存器，連結器也可能使用。

特殊用途暫存器（Special-Purpose Registers）

- **R13（SP，Stack Pointer）**：堆疊指標暫存器，用來管理堆疊操作，記錄堆疊的頂端位置。
- **R14（LR，Link Register）**：連結暫存器，儲存子程式返回位址。在呼叫子程式時，會自動將返回位址存入 `LR`。
- **R15（PC，Program Counter）**：程式計數器，負責控制指令執行位置與流程；正常順序執行會前往下一條指令，分支則改變執行位置。**當 PC 作為指令的來源操作數時，讀值不一定等於目前指令位址**（後續「三階段 Pipeline」以圖解說明）。

AArch32 暫存器分工總覽



閱讀重點： **R0～R3 主要處理參數與暫時資料；R4～R8、R10～R11 需由 Callee 保存；R9 依 ABI 決定，SP / LR / PC 負責堆疊與控制流程。**

程式狀態暫存器（Program Status Registers, PSR）

以下以 AArch32 的 A-profile（Application profile） / R-profile（Real-time profile） 為例：

1. **CPSR（Current Program Status Register）**：記錄條件旗標、處理器模式、執行狀態等。
2. **SPSR（Saved Program Status Register）**：部分例外模式具有對應 SPSR，供例外返回時恢復先前狀態；並非任何特權模式或任何 Arm 核心都有 SPSR。

Cortex-M 使用 xPSR（組合程式狀態暫存器）等狀態暫存器；AArch64（Arm Architecture 64-bit 執行狀態）則以 PSTATE（Processor State）等機制表示處理器狀態。

常用狀態旗標位元

- **N（Negative）**：反映結果的最高位元；有號比較須配合 V 判斷。
- **Z（Zero）**：結果為 0 時設為 1。
- **C（Carry）**：加法進位；減法時 C=1 表示**沒有借位**（no borrow）。
- **V（Overflow）**：有號運算結果超出可表示範圍時設為 1。

`CMP`（Compare）、`ADDS`（Add and Set Flags）、`SUBS`（Subtract and Set Flags）等會更新相關旗標；一般 `ADD`、`SUB` 不一定會更新旗標（視指令形式）。

IPSI Lab, Yuntech-CSIE

Arm 指令集與記憶體存取

資料處理指令（Data Processing Instructions）

- **ADD（Add）**：加法運算。範例：`ADD R0, R1, R2` 將 `R1` 和 `R2` 的值相加，結果存入 `R0`。
- **SUB（Subtract）**：減法運算。範例：`SUB R0, R1, R2` 將 `R1` 減去 `R2` 的值，結果存入 `R0`。
- **MOV（Move）**：賦值指令。範例：`MOV R0, #10` 將數值 `10` 存入 `R0`。
- **MUL（Multiply）**：乘法運算。範例：`MUL R0, R1, R2` 將 `R1` 和 `R2` 的值相乘，結果存入 `R0`。

IPSI Lab, Yuntech-CSIE

位元操作指令（Bitwise Operations）

- **AND（Bitwise AND）**：位元與運算。範例：`AND R0, R1, R2` 將 `R1` 和 `R2` 的值進行位元與，結果存入 `R0`。
- **ORR（Logical OR）**：位元或運算。範例：`ORR R0, R1, R2` 將 `R1` 和 `R2` 的值進行位元或，結果存入 `R0`。
- **EOR（Exclusive OR）**：位元異或運算。範例：`EOR R0, R1, R2` 將 `R1` 和 `R2` 的值進行位元異或，結果存入 `R0`。

IPSI Lab, Yuntech-CSIE

載入和儲存指令（Load and Store Instructions）

- **LDR（Load Register）**：從記憶體載入資料至暫存器。範例：`LDR R0, [R1]` 將 `R1` 指向的記憶體位址內容載入 `R0`。
- **STR（Store Register）**：將暫存器資料儲存至記憶體。範例：`STR R0, [R1]` 將 `R0` 的值儲存到 `R1` 指向的記憶體位址。

IPSI Lab, Yuntech-CSIE

Load/Store：位址與資料是兩回事

假設 `num1` 位於 `0x2000`，記憶體中的 32-bit 整數是 `10`：

指令	執行後的結果	意義
`LDR R0, =num1`	`R0 = 0x2000`	由組譯器處理的**載入位址虛擬指令**
`LDR R1, [R0]`	`R1 = 10`	把位址 `0x2000` 的資料讀到暫存器
`ADD R1, R1, #5`	`R1 = 15`	在暫存器中運算
`STR R1, [R0]`	`Mem[0x2000] = 15`	結果寫回記憶體

核心概念： `R0` 儲存的是地址，`R1` 儲存的是值；`ADD` 本身不會修改記憶體。

此處地址為概念示例；實際連結位址以 ELF（Executable and Linkable Format）/ Linker Script 為準。

靈活定址模式

IPSI Lab, Yuntech-CSIE

LDR/STR 的靈活定址模式：偏移量從哪裡來？

LDR（Load Register）由記憶體**讀入**暫存器；STR（Store Register）將暫存器資料**寫入**記憶體。兩者都需要先算出**有效位址（Effective Address）**。



先掌握第一個問題： `[R1, #4]` 和 `[R1, R2]` 都從 `R1` 加上偏移量；差別只在**偏移是指令中的常數 `#4`，還是來自 R2**。上述都不修改 R1。

LDR/STR 的靈活定址模式：前置 vs 後置索引

② 基底 R1 何時更新？前置 vs 後置索引

每一例都從 R1 = 0x1000 開始；Mem[0x1000] = 10，Mem[0x1004] = 20

前置索引（Pre-indexed）

```
LDR R0, [R1, #4]!
```

0x1000	0x1004	讀入 R0
資料 10	讀取 20	20

R1 執行前 = 0x1000

R1 執行後 = 0x1004

寫回 R1

後置索引（Post-indexed）

```
LDR R0, [R1], #4
```

0x1000	0x1004	讀入 R0
讀取 10	資料 20	10

R1 執行前 = 0x1000

R1 執行後 = 0x1004

寫回 R1

有 !：使用偏移後的位址；偏移寫在] 外：本次使用舊位址。兩者都寫回 R1。

再掌握第二個問題： `[R1, #4]!` 使用**更新後位址**並寫回 R1；`[R1], #4` 本次使用**原 R1 位址**，再寫回 R1。

這描述**指令語意**，不是硬體內部微操作的固定時序。`STR` 也遵循相同定址規則，但資料傳送方向相反。

三種定址模式、四種寫法：從標點符號判斷

模式	寫 法	本次使用的位址	R1 寫回？	判讀
Offset（立即數）	`[R1, #4]`	舊 R1 + 4	否	偏移量在 `[]` 內，無 `!`
Offset（暫存器）	`[R1, R2]`	舊 R1 + R2	否	R2 是 **byte offset**， 不是自動縮放的陣列索引
Pre-indexed	`[R1, #4]!`	舊 R1 + 4	是	`!`：把有效位址寫回 R1
Post-indexed	`[R1], #4`	舊 R1	是	`#4` 在 `]` 外：先用舊址，再更新 R1

重點： 先判斷本次存取的記憶體位址，再判斷基底暫存器 R1 是否更新。**! 代表前置寫回，] 後面的偏移量代表後置更新**。

注意： 這是程式可觀察的**指令語意**，不代表硬體微操作一定依文字所述的時間順序執行。實際上立即數 / 暫存器偏移也能與某些索引形式組合；以上是本章常用的四種寫法。

三種定址：以相同起點追蹤 R1



重點： 都從 `R1=0x1000` 開始；比較「本次從何處讀取」和「R1 是否更新」。後續 buffer、封包與 `*p++` 範例會用到這些模式。

暫存器偏移：陣列索引為何需要 LSL #2？



若 `R1` 是 `int32_t` 陣列基底，`R2` 是**第幾個元素**，便須乘以每個元素的 **4 bytes**。因此以 `LDR R0, [R1, R2, LSL #2]` 存取 `arr[R2]`；原本的 `[R1, R2]` 只把 R2 當位元組偏移，並**不會自動乘 4**。

靈活定址模式：前置/後置索引使用時機

IPSI Lab, Yuntech-CSIE

前置索引使用時機

掃描資料結構（例如 struct、buffer）

讀連續 32-bit 欄位（假設 `R1` 起初指向第一欄位**前方 4 bytes**）：

```
LDR R0, [R1, #4]!     @ 指到欄位 1 並讀取
LDR R2, [R1, #4]!     @ 指到欄位 2 並讀取
LDR R3, [R1, #4]!     @ 指到欄位 3 並讀取
```

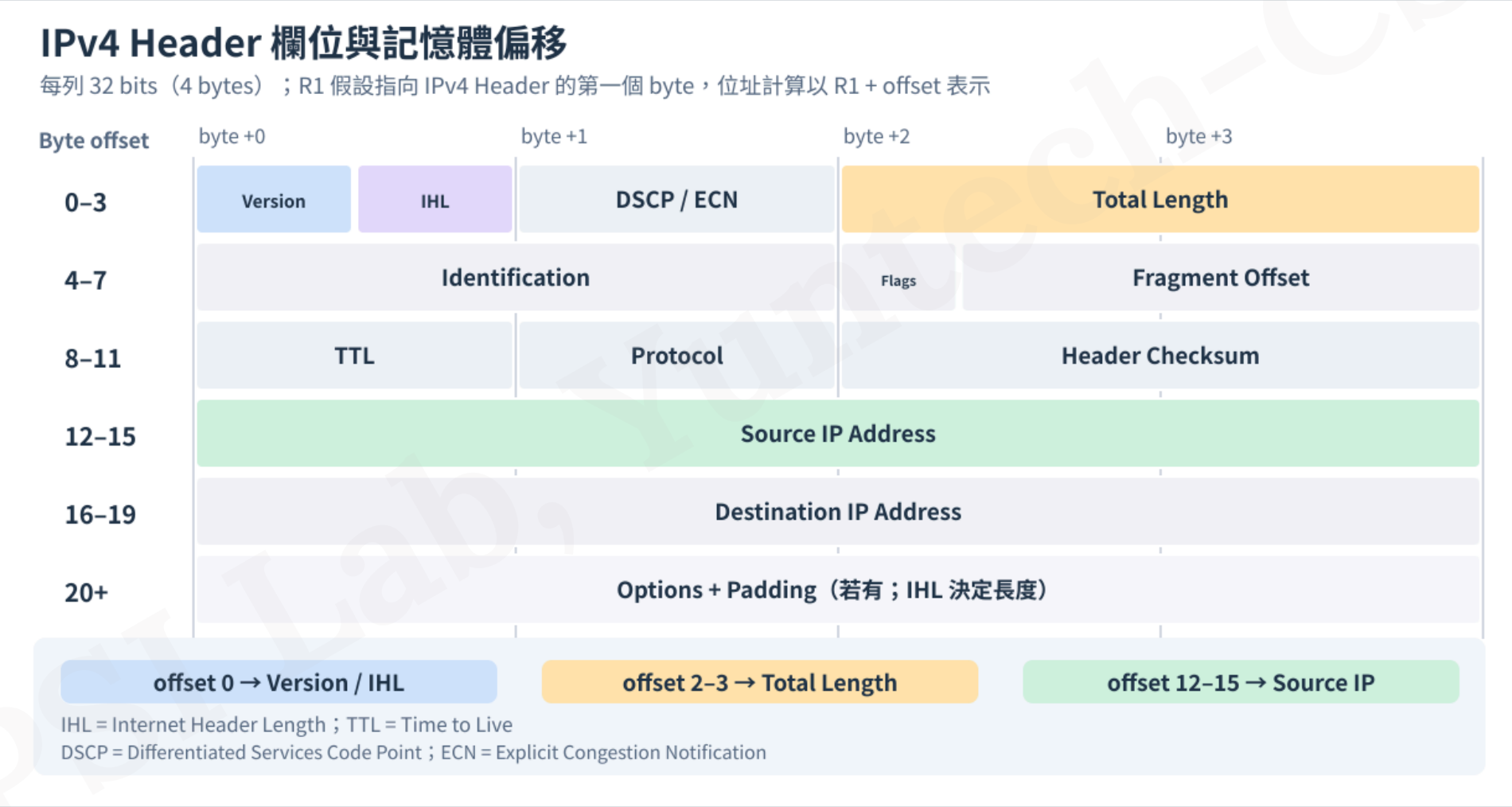
等效的分開寫法（以第一筆為例）：

```
ADD R1, R1, #4
LDR R0, [R1]
```

前置索引可合併位址更新與存取；**是否較快仍須看 CPU 實作**。

IPSI Lab, Yuntech-CSIE

IPv4 Header：欄位位置與 Offset



重點：以 Header 起始位址為基底，下一頁使用 `LDRB` 讀取 offset 0、2、3、12。

IPSI Lab, Yuntech-CSIE

在解碼通訊協定時（逐欄位解析）：前置索引讀取 IPv4 Header

假設 `R1 = 0x1000`，指向 **IPv4 Header 的起始位址**。以下只示範部分欄位：

- **Offset 0**：Version / IHL（Internet Header Length）
- **Offset 2～3**：Total Length（高位元組、低位元組）
- **Offset 12～15**：Source IP Address

使用 `LDRB`（Load Register Byte）每次讀取 **1 byte**；`!` 表示讀取時也將新位址寫回 `R1`。

```
LDRB R0, [R1]          @ 讀 offset 0：Version / IHL；R1 不變
LDRB R2, [R1, #2]!   @ R1 = 0x1002；讀 offset 2：Total Length 高 byte
LDRB R3, [R1, #1]!   @ R1 = 0x1003；讀 offset 3：Total Length 低 byte
LDRB R4, [R1, #9]!   @ R1 = 0x100C；讀 offset 12：Source IP 第一個 byte
```

讀取順序	R1 執行前	本次增加	讀取位址	R1 執行後
第 1 條	`0x1000`	0	`0x1000`	`0x1000`
第 2 條	`0x1000`	+2	`0x1002`	`0x1002`
第 3 條	`0x1002`	+1	`0x1003`	`0x1003`
第 4 條	`0x1003`	+9	`0x100C`	`0x100C`

重點： `#2`、`#1`、`#9` 是**相對目前 R1** 的偏移量，不是一直相對 Header 起點。最後使用 `#9`，是因為 R1 已在 offset 3，距離 offset 12 還有 9 bytes。

IPv4 範例延伸：將讀到的 bytes 解讀為欄位值

接續上一頁，假設讀到：`R0 = 0x45` 、`R2 = 0x00` 、`R3 = 0x3C` 、`R4 = 0xC0` 。

1. Version 與 IHL：同一個 byte 的高、低 4 bits

`0x45` = `0100 0101`（二進位）：高 4 bits 是 Version，低 4 bits 是 IHL。

```
MOV R5, R0, LSR #4       @ 高 4 bits：Version = 4
AND R6, R0, #0xF         @ 低 4 bits：IHL = 5
```

- **Version = 4**：代表 IPv4。
- **IHL = 5**：單位為 4 bytes，因此 Header 長度 = `5 × 4 = 20 bytes` 。

2. Total Length：依 Network Byte Order（big-endian）合併兩個 bytes

```
ORR R2, R3, R2, LSL #8  @（高 byte << 8）| 低 byte
```

- 高 byte `0x00`、低 byte `0x3C`，合併為 `0x003C = 60 bytes`。
- **Total Length** 包含 IPv4 Header 與資料內容；`R4 = 0xC0` 則是來源 IP 的第一個 byte（十進位 192），並非完整來源位址。

重點： `LDRB` 與定址模式回答「**從哪裡讀？**」；`LSR`、`AND`、`ORR` 及位元組序回答「**讀出的值代表什麼？**」。

註：此處是指令教學片段；若寫成供 C 呼叫的函式，使用 R4～R6 時，應依 AAPCS32 保存並恢復其原值。*

後置索引使用時機

實作「讀資料後才遞增」的情境（類似 C 的 *p++）

C 語言：

```
x = *p++;
```

假設 `p` 的型別為 `uint32_t *`，可能編譯為：

```
LDR R0, [R1], #4        @ 讀原位置，再遞增 4 bytes
```

因為語意是 `*(p++)`：先使用原位置，再更新指標。實際指令取決於編譯器最佳化；如果 `p` 是 `uint8_t *`，增量則通常是 1 byte。

IPSI Lab, Yuntech-CSIE

讀取一塊記憶體 buffer（如影像、音訊）時要走訪「下一筆」

需要用到目前這筆資料。

讀目前像素 → 然後往右走：

```
LDR R3, [R0], #4    @ 讀當前 32-bit 像素，R0 再移到下一筆
```

用於：

- 影像逐像素處理（**每像素 4 bytes** 的格式）
- 32-bit 音訊 sample 串列
- 32-bit 感測器資料流解析

> **如果資料元素不是 4 bytes，位址增量與 LDR / LDRB / LDRH 指令也需調整。**

IPSI Lab, Yuntech-CSIE

Arm 程式控制與函式呼叫

分支指令（Branch Instructions）

IPSI Lab, Yantech-CSIE

分支指令（Branch Instructions）

Arm 的分支指令不僅用於一般跳躍，也扮演 函式呼叫、流程控制（if/loop）、模式切換（ARM/Thumb） 的關鍵角色。

- **B（Branch）**：無條件跳躍。範例：`B label` 跳至 `label` 指定的位置。
- **BL（Branch with Link）**：分支並儲存返回位址，用於子程式呼叫。範例：`BL subroutine` 跳至 `subroutine`，返回位址儲存在 `LR`。
- **BX（Branch and Exchange）**：依暫存器指定的目標進行間接跳躍，並依最低位元選擇 ARM/Thumb 狀態；`BX LR` 常用於子程式返回。

B（Branch，無條件跳躍）

```
B label
```

功能：

- 立即跳到 `label` 指定的程式位置（PC ← label）。
- 不會保存返回位址。

常見用途：

- while/for 迴圈跳躍
- if/else 流程控制
- 實作無窮迴圈（例如裸機程式的 main 無結束點）

範例：

```
loop:
    ADD R0, R0, #1
    B loop          @ 無條件跳回 loop
```

BL（Branch with Link，分支並連結）

```
BL subroutine
```

功能：

- 跳到 `subroutine` 執行
- **同時把返回位址存進 LR（R14）**

BL 的核心：BL 是 A32（傳統 ARM）指令集中的「function call」。

IPSI Lab, Yuntech-CSIE

範例：

```
BL add_func    @ 呼叫 add_func
```

BL 呼叫時的概念（以下為示意，不是可組譯指令）：

```
LR ← BL 後續指令的返回位址資訊
PC ← add_func 的目標位址
```

A32 與 T32 對「讀取 PC」的定義不同，不能概括成 `LR = PC + 4`。對典型 A32 `BL`，保存的是該 BL 下一條指令的返回位址資訊；這與 A32 讀取 PC 可見值為指令位址 +8 不矛盾。

IPSI Lab, yuntech-CSIE

BX（Branch and Exchange）：函式返回與 ARM / Thumb 切換

```
BX  LR                  @ 跳至 LR 指定的位置，常用於函式返回
BX  R3                  @ 跳至 R3 指定的位置，不更新 LR
```

功能：

- BX 從暫存器取得跳躍目標，並依目標值的最低位元（bit 0）決定執行 **ARM（A32）** 或 **Thumb** 指令集。

用途：

1. **函式返回**：`BX LR`，回到呼叫端。
2. **ARM / Thumb 狀態切換**：依目標值的 bit 0 選擇指令集狀態。
3. **間接跳躍（Indirect Branch）**：`BX R3`，目標由 R3 的內容決定，而非寫死在 `B label`。

目標暫存器值	bit 0	跳躍後狀態	實際取指位址
`0x8000`	0	ARM	`0x8000`
`0x8001`	1	Thumb	`0x8000`

重點： **bit 0 是指令集狀態標記，不代表 Thumb 指令從奇數位址開始。BX 不一定真的切換狀態，也不會自動轉換指令內容；這裡不是特權模式或 AArch32 / AArch64 的切換。**

IPSI Lab, Yuntech-CSIE

BX 與 BLX：間接跳躍與函式指標（Function Pointer）

C 語言的函式指標可在執行期間決定要呼叫哪個函式：

```c
extern int add_numbers(int a, int b);
int demo(void) {
    int (*operation)(int, int) = add_numbers;
    return operation(5, 3);  // 經由函式指標呼叫
}
```

以 A32 表達相同呼叫概念（編譯器不一定產生完全相同的指令）：

```
MOV R0, #5              @ 第 1 個參數
MOV R1, #3              @ 第 2 個參數
LDR R3, =add_numbers    @ 將函式目標位址資訊放入 R3
BLX R3                  @ 間接呼叫，並將返回位址資訊存入 LR
```

IPSI Lab, Yuntech-CSIE

指令	跳躍目標來源	更新 LR？	典型用途
`BX LR`	LR	否	函式返回
`BX R3`	R3	否	間接跳躍、尾端跳躍
`BLX R3`	R3	是	透過函式指標呼叫

重點： `BX` 是「跳躍」，**不保存新的返回位址**；`BLX` 是「帶連結的跳躍」，**會設定 LR**，適合之後需要返回的間接函式呼叫。兩者均可依目標 bit 0 選擇 ARM / Thumb 狀態。

IPSI Lab, Yuntech-CSIE

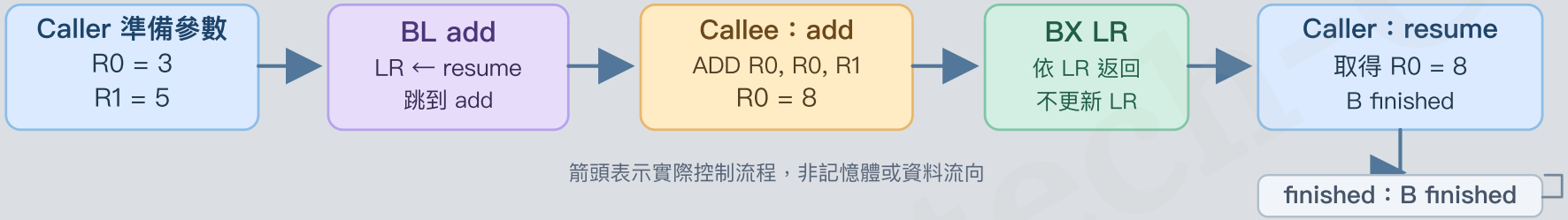
BL、LR、BX：完整的呼叫與返回

以下示範 Caller（呼叫者） 傳入兩個整數，呼叫 Callee（被呼叫函式） `add` ，並取得返回值。

```
    MOV R0, #3              @ 第 1 個參數 a = 3
    MOV R1, #5              @ 第 2 個參數 b = 5
    BL  add                 @ LR 記錄 resume 的返回位址，跳至 add
resume:
    @ 回到 Caller，此時 R0 = 8
    B   finished            @ 避免程式直接往下執行到 add
add:
    ADD R0, R0, R1          @ 計算 3 + 5，結果放回 R0
    BX  LR                  @ 依 LR 返回 resume
finished:
    B   finished            @ Bare-metal 示意：持續停在此迴圈
```

重點： `BL add` 會設定 LR 並跳至 `add` ；`BX LR` 依 LR 返回 `resume` 。此例的 `add` 是 **Leaf Function（葉函式）**，未再呼叫其他函式，也未修改需要保存的暫存器，因此不必額外 `PUSH` / `POP` 。

BL、LR、BX：完整的呼叫與返回（流程對照）



步驟	執行位置 / 指令	結果與控制流程
1. 傳入參數	`MOV R0, #3` 、 `MOV R1, #5`	R0 = 3、R1 = 5
2. 函式呼叫	`BL add`	**LR ←** `resume` ，轉至 `add`
3. 執行運算	`ADD R0, R0, R1`	**R0 = 8**，LR 不變
4. 函式返回	`BX LR`	轉回 `resume` ，Caller 取得 R0 = 8
5. 後續流程	`B finished` → `B finished`	避免落入 `add` ，停在 Bare-metal 示意迴圈

為何下一節需要 AAPCS32？ `BL` / `BX` **解決「跳到哪裡、如何返回」；AAPCS32 進一步約定參數與返回值放在哪些暫存器，以及跨呼叫應保存哪些暫存器。**

函式呼叫標準 (AAPCS)

IPSI Lab, Yuntech-CSIE

函式呼叫標準（AAPCS32）

- **AAPCS32（Procedure Call Standard for the Arm Architecture，32-bit 版本）**：規範 AArch32 函式間的**參數傳遞、返回值、暫存器保存與堆疊使用**。
- 前一單元的 `BL` 、 `BX LR` 已解釋**如何呼叫與返回**；本單元進一步回答：**Caller 與 Callee 如何合作，才不會破壞彼此需要的資料？**
- 以下先討論一般 **32-bit 整數參數**；結構、較大資料型別與浮點參數有其他配置規則。

核心概念： **AAPCS32 是 Caller（呼叫者）與 Callee（被呼叫者）共同遵守的呼叫介面契約，不是新的 CPU 指令。**

AAPCS32：Caller / Callee 的暫存器分工

暫存器	常見用途	誰負責保留原值？
R0–R3	一般參數、部分返回值、暫存	**Caller** 若呼叫後仍需要舊值，須自行保存
R4–R8、R10–R11	函式區域資料	**Callee** 若修改，返回前須恢復
R9	平台暫存器或變數暫存器	依平台 **ABI**（Application Binary Interface）決定
R12（IP）	暫時資料、連結器輔助	不保證跨呼叫保留
R13（SP）	Stack Pointer（堆疊指標）	Callee 返回時須恢復適當堆疊狀態
R14（LR）	Link Register（返回位址資訊）	若將被其他 `BL` 覆寫，須先妥善保存

對齊規則： AAPCS32 要求 `SP` 在一般使用時至少 **4-byte 對齊**；在公開函式介面必須 **8-byte 對齊**（`SP mod 8 = 0`）。

AAPCS32：Caller 與 Callee 如何協作？



閱讀重點： Caller 安排參數與必要的暫存器保存；Callee 負責保留其必須維持的暫存器與堆疊狀態，並把一般整數返回值放入 `R0`。

A. 呼叫者（Caller）的工作

1. 安排參數並呼叫函式（以一般 32-bit 整數為例）：

參數	第 1 個	第 2 個	第 3 個	第 4 個
暫存器	`R0`	`R1`	`R2`	`R3`

```
MOV R0, #3          @ 第 1 個參數 a = 3
MOV R1, #5          @ 第 2 個參數 b = 5
BL  add             @ 呼叫 add(a, b)
@ 返回後，R0 = 8
```

2. 保存需要的舊值： `R0–R3` 、 `R12` 不保證跨呼叫維持原值；若仍需使用，Caller 應事先安排保存，不是每次都要 PUSH。

3. 準備 Stack： 超出可使用暫存器的參數，依 AAPCS32 的型別與對齊規則配置於 Stack；呼叫公開函式時確保 `SP` 為 8-byte 對齊。

B. 被呼叫者（Callee）的工作

Callee 的基本流程是：接收參數 → 必要時保存狀態 → 執行運算 → 放置返回值 → 恢復狀態並返回。

- **接收與運算：** 讀取 `R0–R3` 中的一般整數參數；一般 32-bit 整數結果由 `R0` 返回。
- **保存責任：** 若修改 `R4–R8`、`R10–R11`，返回前須恢復；`R9` 依平台 ABI 處理。
- **LR 與 Stack：** 若還會執行其他 `BL`，必須在原 LR 被覆寫前保存所需返回資訊，並在呼叫邊界維持正確對齊。

例如需要保存 `R4` 與 `LR` 時，可成對使用：

```
PUSH {R4, LR}     @ 共保存 8 bytes
@ ...執行運算，必要時呼叫其他函式...
POP  {R4, LR}     @ 還原原值
BX   LR          @ 返回 Caller
```

注意：以上是保存與恢復的流程片段，並非每個函式都要使用 `PUSH/POP`；是否需要保存取決於實際使用情況。

完整執行範例：C 與 Assembly 對照

C 語言的函式：

```c
int add(int a, int b) {
    return a + b;
}
```

A32 組合語言中的 Caller / Callee（Bare-metal 流程示意）：

```
    MOV R0, #3          @ Caller：a = 3
    MOV R1, #5          @ Caller：b = 5
    BL  add             @ 呼叫 add，LR 保存返回位置
resume:
    B   finished        @ 使用返回值 R0 = 8 後，離開呼叫流程
add:
    ADD R0, R0, R1      @ Callee：回傳 a + b = 8
    BX  LR              @ 返回 resume
finished:
    B   finished        @ Bare-metal 示意：停在此處
```

IPSI Lab, Yuntech-CSIE

AAPCS32：為什麼需要函式呼叫協定？

1. **參數與返回值有共同規則：** Caller / Callee 知道資料應放在哪些暫存器或 Stack 位置。
2. **暫存器保存責任明確：** 函式呼叫不會任意破壞另一方依規範應能保留的資料。
3. **堆疊使用一致：** 呼叫邊界的對齊與堆疊恢復方式有明確要求。
4. **支援 C / Assembly 模組協作：** 符合相同目標架構與 ABI 的程式模組可互相呼叫；不代表可以直接跨不同 CPU 執行程式。

銜接下一單元： 如果 Callee 又呼叫其他函式，或遞迴呼叫自己，新的 `BL` 會覆寫 `LR`。接下來透過 **Leaf / Non-leaf Function** 與 `factorial()`，理解為什麼要保存返回位址及如何使用 Stack。

巢狀函式呼叫與遞迴
(Nested Calls and Recursion)

IPSI Lab, NTUTech-CSIE

Leaf 與 Non-leaf Function：何時需要保存 LR？

前面已學過 `BL` 會將返回位址資訊寫入 `LR`，`BX LR` 可用來返回。如果函式執行途中又呼叫其他函式，原本的 LR 該如何保留？

- **Leaf Function（葉函式）**：函式內不再呼叫其他函式；若未修改需要保存的暫存器，通常可直接以 `BX LR` 返回。
- **Non-leaf Function（非葉函式）**：函式內會呼叫其他函式，或遞迴呼叫自己。新的 `BL` 會覆寫 `LR`；若之後還要返回上一層，就須先保留原返回位址。

Leaf Function 範例

Leaf Function不再呼叫其他函式。例如 C 程式：

```c
int add(int a, int b) { return a + b; }
```

對應的簡單 A32 函式：

```
add:
    ADD R0, R0, R1      @ 參數來自 R0、R1；結果留在 R0
    BX  LR              @ 未再呼叫其他函式，無須額外保存 LR
```

重點： Leaf / Non-leaf 的區別是「函式內有沒有再呼叫其他函式」，**不是有沒有使用 Stack**。下一頁改用遞迴 `factorial()` 說明為什麼必須保存每一層的 LR。

IPSI Lab, Yuntech-CSIE

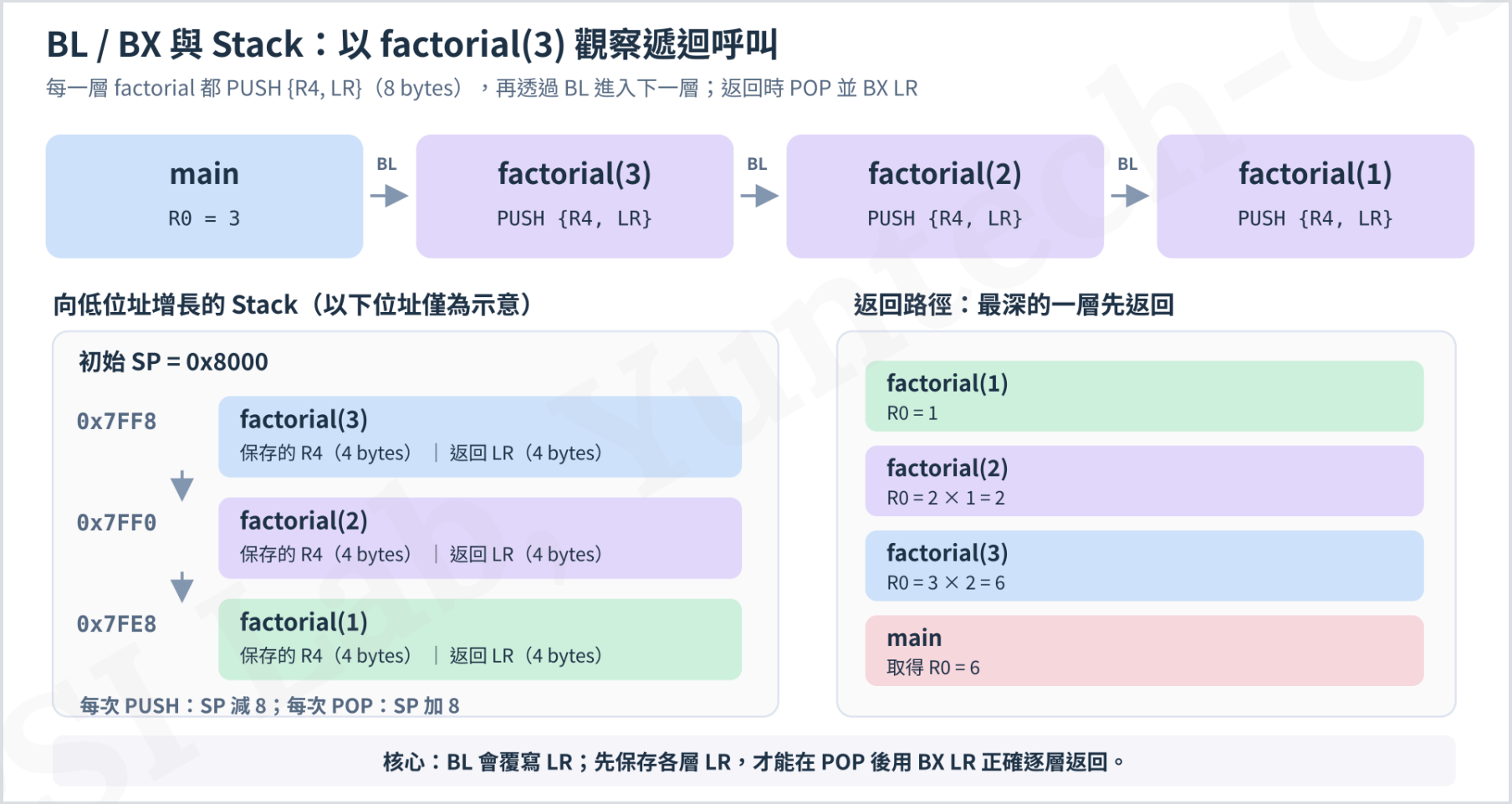
Non-leaf Function 範例：遞迴 `factorial()`

以 `R0 = n` 呼叫 `factorial(n)`；本例假設輸入為 0～12 的整數，其階乘結果可放入 32-bit 有號整數，且涵蓋 `0! = 1`。

```
factorial:
    PUSH {R4, LR}          @ 保存本層 R4、LR（共 8 bytes）
    MOV  R4, R0            @ R4 記住本層的 n
    CMP  R0, #1
    BGT  recurse           @ n > 1 時才進入遞迴
    MOV  R0, #1            @ n = 0 或 1：結果為 1
    B    done
recurse:
    SUB  R0, R0, #1
    BL   factorial         @ 遞迴呼叫：會覆寫目前 LR
    MUL  R0, R4, R0        @ R0 = n × factorial(n-1)
done:
    POP  {R4, LR}          @ 還原本層保存的 R4 與 LR
    BX   LR                @ 返回上一層
```


重點： 每進入一層都以 `PUSH {R4, LR}` 保存自己的資料；回程以 `POP` 還原，再由 `BX LR` 逐層返回。這段程

遞迴時 Stack 如何保住每一層的 LR？



重點 / 連到後續 GDB（GNU Debugger）： 圖中以 `factorial(3)` 示範每層 `PUSH` 使 `SP` 減少 8 bytes，`POP` 恢復本層的 `R4` / `LR`，再以 `BX LR` 返回。後續可用 `factorial(5)` 追蹤 `SP`、`LR`、`R0` 如何逐層變

條件執行指令

IPSI Lab, Yuntech-CSIE

條件執行：A32、T32 與 A64 的設計差異

程式需要依條件決定「是否執行指令、是否跳躍，或選擇哪個結果」。這三種指令集都能表達條件邏輯，但方式不同。

指令集	條件控制方式	典型指令
A32（AArch32）	多數一般指令可直接附加條件後綴	`MOVEQ`、`ADDNE`、`BGT`
T32（AArch32）	條件分支；支援 Thumb-2 的核心可用 `IT` 控制後續指令	`BEQ`、`IT EQ`
A64（AArch64）	不採 A32 式廣泛條件後綴，改用專用條件指令	`B.EQ`、`CSEL`、`CCMP`

先區分架構： **AArch32、AArch64 是執行狀態；A32、T32、A64 是指令集。Armv8-A 不等於只有 A64，是否支援 AArch32 仍須看實作。**

縮寫：IT = If-Then；CSEL = Conditional Select；CCMP = Conditional Compare。

A32：條件碼如何控制指令？

在 A32 中，多數資料處理、記憶體存取及分支指令可附加條件後綴；是否執行由 NZCV 旗標決定。

條件後綴	判斷條件	範例與意義
`EQ` (Equal)	Z = 1	`MOVEQ R0, #1`：相等才移入 1
`NE` (Not Equal)	Z = 0	`ADDNE R1, R1, #1`：不相等才加 1
`GT` (Greater Than，Signed)	Z = 0 且 N = V	`BGT label`：有號大於才分支
`HI` (Higher，Unsigned)	C = 1 且 Z = 0	`BHI label`：無號大於才分支

- NZCV：在 AArch32 為 CPSR 的條件旗標；AArch64 則為 PSTATE.NZCV。
- `CMP` 依減法設定旗標，不把相減結果寫回暫存器；A32 中一般的 `ADD`、`MOV`（未加 `S`）不會更新旗標。

重點： 指令後綴決定檢查哪個條件；旗標本身來自前面的比較或運算。

A32 範例：不使用分支的 `if`

C 語言：

```
if (x == 0)
    y = y + 1;
```

A32 組合語言（R0 = `x`，R1 = `y`）：

```
CMP   R0, #0          @ 比較 x 與 0，更新 NZCV
ADDEQ R1, R1, #1      @ 只有 Z=1 (x==0) 才執行
```

執行過程：

1. `CMP` 判斷 `x` 是否為 0；若相等，設定 Z = 1。
2. `ADDEQ` 僅在 Z = 1 時增加 R1；否則略過這條指令。
3. 這個簡短 `if` **不需要額外的條件分支**，但不代表在所有處理器上一定比較快。

本例是 **A32** 語法；不能直接寫成 A64 的 `ADDEQ X1, X1, #1`。

Signed / Unsigned：同一次 `CMP`，為何判斷不同？

- **Signed（有號整數）** 與 **Unsigned（無號整數）** 是同一組位元的兩種數值解讀方式。例如 32-bit 的 `0xFFFFFFFF`，有號時是 –1，無號時是 4,294,967,295。
- Arm 的 CMP 會更新 NZCV 旗標，後續使用 BGT、BLT 等進行有號比較，或使用 BHI、BLO 等進行無號比較。

IPSI Lab, Yuntech-CSIE

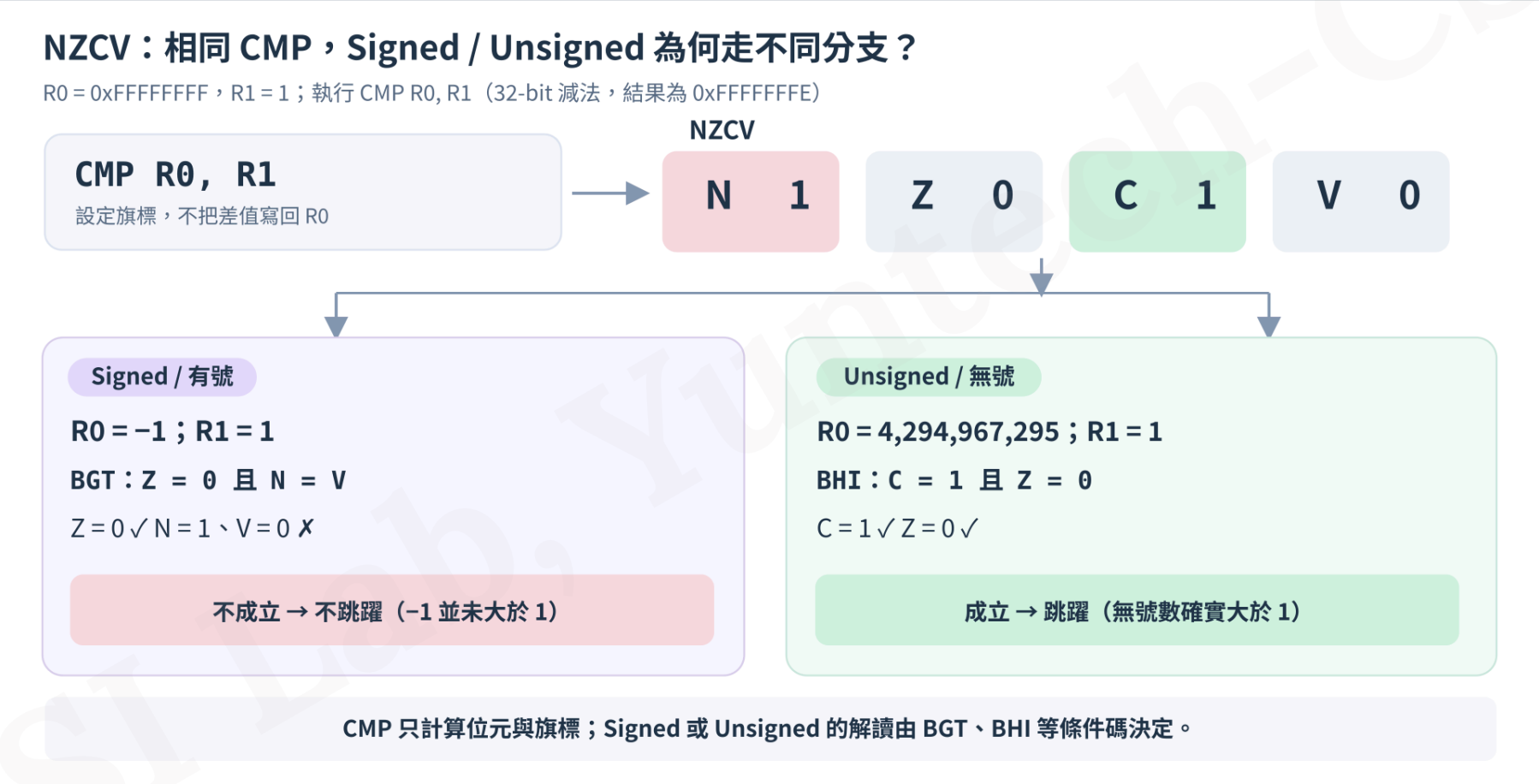
```
MVN R0, #0                  @ R0 = 0xFFFFFFFF
MOV R1, #1
CMP R0, R1                 @ R0 – R1：更新 NZCV，不儲存相減結果
BGT signed_greater          @ 有號大於？不成立
BHI unsigned_higher         @ 無號大於？成立
```

判斷方式	R0 的解讀	比較 `R0 > 1`	結果
Signed（GT）	–1	–1 > 1	不跳躍
Unsigned（HI）	4,294,967,295	4,294,967,295 > 1	跳躍

關鍵： `CMP` **不知道 C 變數的型別；選用** `GT` **或** `HI` **等條件碼，才決定採有號或無號的比較方式。**

以上為判斷語意片段，未列出分支目標程式。

用 NZCV 追蹤有號 / 無號分支



閱讀重點： 這次 `CMP R0, R1` 得到 **N=1、Z=0、C=1、V=0**。因此 `BGT` 的 `N=V` 不成立；`BHI` 的 `C=1` 且 `Z=0` 成立。

A64：不用 A32 式的通用條件後綴

- **A64（AArch64）不允許直接把 `EQ`、`NE` 等條件碼加在多數一般指令後面：**

```
ADDEQ X0, X1, X2      // 錯：A64 不支援這種寫法
MOVNE X3, #1          // 錯：A64 不支援這種寫法
```

- **A64 改用以下三類專門指令：**

類型	用途	例子
`B.cond`	條件成立才跳躍	`B.EQ label`、`B.GT label`
`CSEL`	不跳躍而選擇兩個來源之一	`CSEL X2, X0, X1, GT`
`CCMP` / `CCMN`	根據先前條件，選擇進行比較或設定替代旗標	`CCMP W0, #0, #0, EQ`

不要混淆： A64 是 AArch64 使用的指令集，但**每條 A64 指令的編碼長度仍為 32 bits**。此處的 A64 指令不能在本課程既有的 ARM926EJ-S / `versatilepb` 環境直接執行。

A64：B.cond 與 CSEL

1. B.cond（條件式分支）：根據先前設定的 NZCV 決定是否跳躍。

```
CMP  W0, #0
B.EQ equal_zero        // W0 == 0 時跳躍
```

2. CSEL（Conditional Select，條件式選擇）：從兩個來源選一個，不需要跳躍。

```
CMP  W0, #0
CSEL W2, W0, W1, GT    // W0 > 0 選 W0；否則選 W1
```

相當於 C 語言：

```
result = (x > 0) ? x : fallback;
```

重點： `CSEL` 的條件已由前一條 `CMP` 設定；`W` 是 AArch64 暫存器的 32-bit 視圖，適合對照此處的 32-bit `int` 範例。另有 `CSINC`、`CSINV`、`CSNEG` 等變形。

A64：`CCMP` 為什麼能串接兩個比較？

目標： 只有當 `a == 0` 而且 `b == 0` 時才跳躍（`W0=a`、`W1=b`）。

```
if (a == 0 && b == 0) {
    // 執行 both_zero 對應的處理
}
```

```
CMP  W0, #0                // ① 比較 a 與 0，設定 NZCV
CCMP W1, #0, #0, EQ        // ② 依第一個比較結果，決定如何設定 NZCV
B.EQ both_zero             // ③ 最後 Z=1 才跳至 both_zero
```

IPSI Lab, Yuntech-CSIE

`CCMP W1, #0, #0, EQ` 的四個運算元：

運算元	在這個範例中的作用
`W1`	第二個要檢查的數值 `b`
第一個 `#0`	當條件成立時，將 `b` 與 **0 比較**
第二個 `#0`	當條件不成立時，將 **NZCV 設成** `0000`
`EQ`	只有前一次比較得到 **Z=1（a=0）**，才比較 `b`

重點： `CCMP` 本身**不會跳躍**。它負責決定下一條 `B.EQ` 要檢查的旗標；兩個 `#0` 的用途不同。

IPSI Lab, Yuntech-CSIE

A64：`CCMP` 的兩種路徑與最後的跳躍結果

`CMP W0, #0` 先檢查 `a`；接著 `CCMP` 根據 **Z 是否為 1** 分成兩條路：

- **若 `a = 0`（EQ 成立）**：真正比較 `b` 與 0，更新 NZCV。只有 `b=0` 才得到 `Z=1`。
- **若 `a ≠ 0`（EQ 不成立）**：**不比較 `b`**，直接將 NZCV 設為 `0000`，其中 `Z=0`，確保 `B.EQ` 不跳躍。

a（W0）	b（W1）	CCMP 做什麼？	最後 Z	B.EQ both_zero
0	0	比較 b=0	**1**	**跳躍**
0	5	比較 b≠0	0	不跳躍
5	0	不比較 b，NZCV ← `0000`	0	不跳躍
5	5	不比較 b，NZCV ← `0000`	0	不跳躍

為何替代值選 `#0` ？ 因為 `B.EQ` 只根據 `Z=1` 判斷是否跳躍；將替代旗標設成 `0000`（`Z=0`），就能讓第一個條件失敗時，整個 `a == 0 && b == 0` 判斷為假。

核心觀念： 一般 `CMP` 直接產生比較旗標；`CCMP` 則是「**條件成立就比較，否則寫入指定的備用 NZCV**」。它可在兩次比較之間避免一條條件分支，但不保證所有處理器上都一定比較快。

IPSI Lab, Yuntech-CSIE

相同的 C 邏輯：A32 與 A64 如何實作？

C 語言（以 32-bit 有號整數為例）：

```
result = (x > 0) ? x : fallback;
```

A32（R0=x，R1=fallback，R2=result）	A64（W0=x，W1=fallback，W2=result）
`CMP R0, #0`	`CMP W0, #0`
`MOVGT R2, R0`	`CSEL W2, W0, W1, GT`
`MOVLE R2, R1`	不需另一條 MOV

- **A32**：以帶條件後綴的 `MOVGT` / `MOVLE` 執行二選一。
- **A64**：以一條 `CSEL` 表達相同的選擇動作（前面仍需 `CMP`）。
- 這裡比較的是**指令集如何表達條件邏輯**，不能據此認定 A64 一定比較快。

A64 為什麼改用專用條件指令？

這是 A32 與 A64 在指令集設計上的取捨，不是「Armv7 有條件執行、Armv8 沒有」。

1. **A32** 的廣泛條件後綴能讓部分短判斷不需要分支，但會占用指令編碼空間並增加設計限制。
2. **A64** 以 `B.cond`、`CSEL`、`CCMP` 等專用指令處理條件控制與比較。
3. 是否使用分支或條件選擇，以及哪種方式較有效率，仍由**處理器微架構、分支預測、編譯器與工作負載**共同決定。

本節總結： `CMP` **產生條件旗標；Signed / Unsigned 由條件碼決定解讀方式；A32 與 A64 則採不同的條件指令設計。下一單元的 Pipeline 與 Branch Prediction 將進一步解釋效能上的取捨。**

Arm 管線（Pipeline）設計

Arm 管線（Pipeline）設計

- **Pipeline（管線）**：將指令處理工作分成階段，讓不同指令在同一個時鐘週期內，分別使用不同階段的硬體。
- **Clock Cycle（時鐘週期）**：同步管線工作的時間節奏；一個時鐘週期內，數個階段可以**同時**處理不同指令。
- **Pipeline Depth（管線深度）**：一條指令須經過的階段數；**階段數不是CPU核心數，也不等於每週期完成的指令數**。

本小節的示意圖均先假設每階段花 1 cycle，沒有 Stall（停頓）或 Flush（清除錯誤路徑）。 真實 CPU 的記憶體延遲、分支與複雜運算可能使指令花費更多週期。

Arm 的三階段管線

早期部分 Arm 處理器將指令處理分成三個主要階段：

1. **Fetch（取指）**：從指令記憶體取得接下來要執行的指令。
2. **Decode（解碼）**：判斷指令要做什麼操作，以及需要哪些暫存器。
3. **Execute（執行）**：讀取所需操作數、執行運算或資料存取，並在適當時機寫回結果。

重點： 不必等指令 I1 三個階段都做完，才開始 I2；當 I1 正在 Execute，I2 可以 Decode，I3 可以 Fetch。

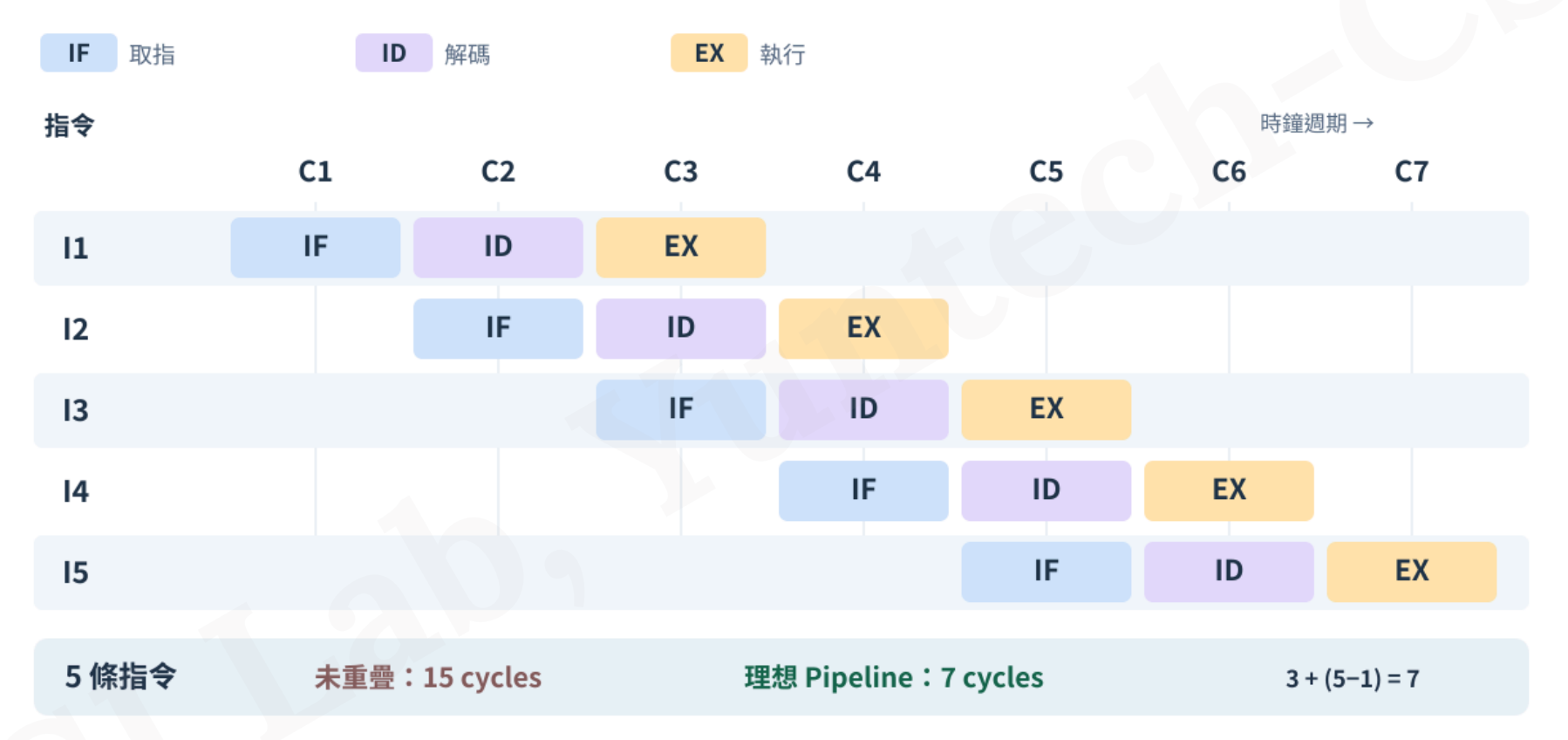
實際案例： ARM7TDMI 採三階段管線，實作的是 **Armv4T**，型號中的「ARM7」不等於「Armv7 架構」。階段數由**核心微架構**決定，不由 ISA（Instruction Set Architecture，指令集架構）版本直接決定。

三階段 Pipeline：各階段做什麼？



閱讀重點： IF → ID → EX 是同一條指令的處理順序；但在執行過程中，不同指令可以重疊占用這三個階段。三階段的 Execute 涵蓋部分存取與寫回工作，不等同於後面五階段模型的 EX 階段。

三階段管線執行範例：5 條指令



- 未使用階段重疊：5 條指令 × 3 cycles ＝ **15 cycles**。
- 理想三階段 Pipeline：3 + (5 − 1) ＝ **7 cycles**。
- 管線填滿後，理想上**每個 cycle 完成一條**指令；不是一個 cycle 完成三條。

Clock Cycle：同一個週期可以做幾件事？



重點： 只有一個 CPU Core，仍可在同一個 Clock Cycle 讓 I1、I2、I3 分別處於 EX、ID、IF；但在這個**單發射三階段模型**中，每週期理想上只完成 **1 條指令**。

Clock Frequency（時脈頻率）：由哪個階段決定？

- 假設 CPU 時脈為 **1 GHz**：一個 Clock Cycle 為 **1 ns**，整個同步管線都以此節奏推進。
- 不代表每個階段都「各有一個 1 GHz 時脈」，而是各階段必須在允許的週期時間內完成其工作。

階段	假設的邏輯延遲	在設計時的意義
Fetch	0.5 ns	不是最慢階段
Decode	0.7 ns	不是最慢階段
Execute	0.8 ns	此例的最慢階段

重點： 時鐘週期須能容納**最慢的關鍵路徑**，還要加上管線暫存器與時序裕量。因此表中的延遲只是示意；如果其餘時序開銷太大，**仍不一定能跑到 1 GHz**。

A32 讀取 PC：先分清楚三個位址

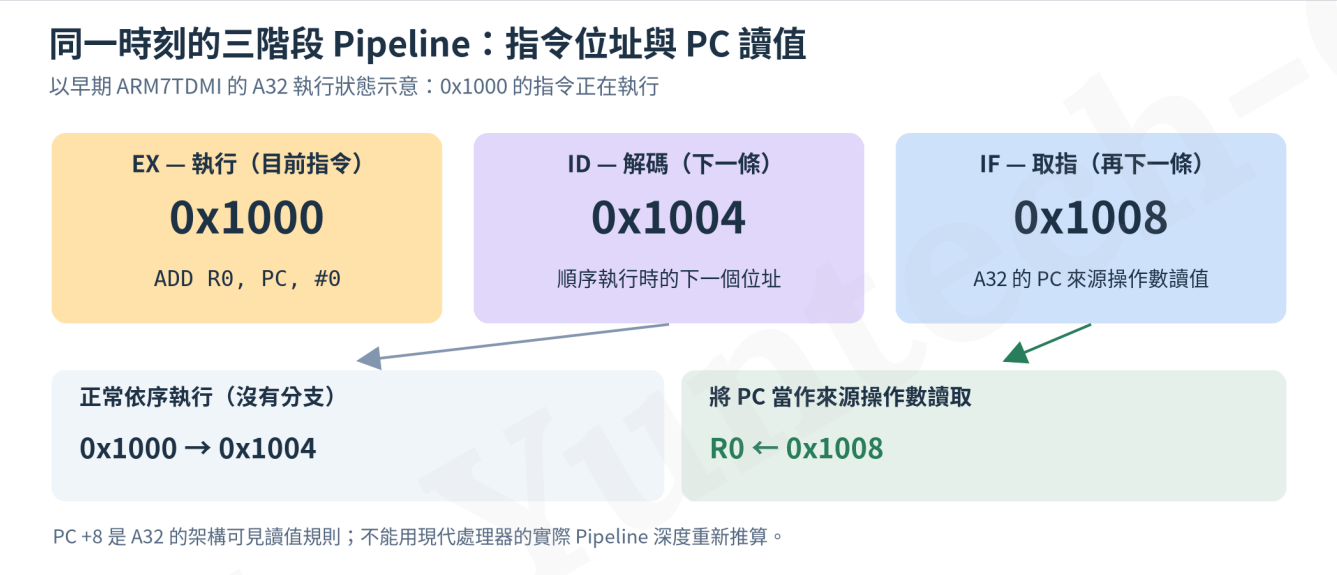
假設有三條 A32 指令（每條 4 bytes），第一條位於 `0x1000`：

```
0x1000: ADD R0, PC, #0     @ 將 PC 的架構讀值存入 R0
0x1004: ADD R1, R1, #1     @ 正常執行的下一條
0x1008: SUB R2, R2, #1
```

問題	位址
目前正在執行哪一條？	`0x1000`
若沒有分支，下一條執行哪一條？	`0x1004`
第一條指令將 PC 當作來源操作數時，讀到多少？	**`0x1008`**

執行 `ADD R0, PC, #0` 後，`R0 = 0x1008`；但 CPU 並沒有因此跳過 `0x1004`！

A32 PC +8：為何會與早期 Pipeline 有關？



- 早期三階段管線的典型情況：`0x1000` 在 **(Execute，正在執行)**，`0x1004` 在 **(Decode，已經進入解碼階段)**，`0x1008` 在 **(Fetch，正在取指階段)**。
- 在早期 ARM7TDMI 的設計中，PC 對應的是取指中的指令位置，而不是正在 Execute 的指令位置。因此，執行中的指令讀到的 PC 值會超前兩條 A32 指令，也就是 8 bytes。
- 因此，歷史上 A32 指令讀取 PC，會看到「**目前執行指令位址 +8**」的值。

重要區分： `PC +8` **已成為 A32 的指令集架構規則，不是按照現代處理器實際有多少級 Pipeline 重新計算。T32 的一般 PC 相關讀值通常是目前指令位址 +4，部分 PC-relative 操作還涉及對齊。**

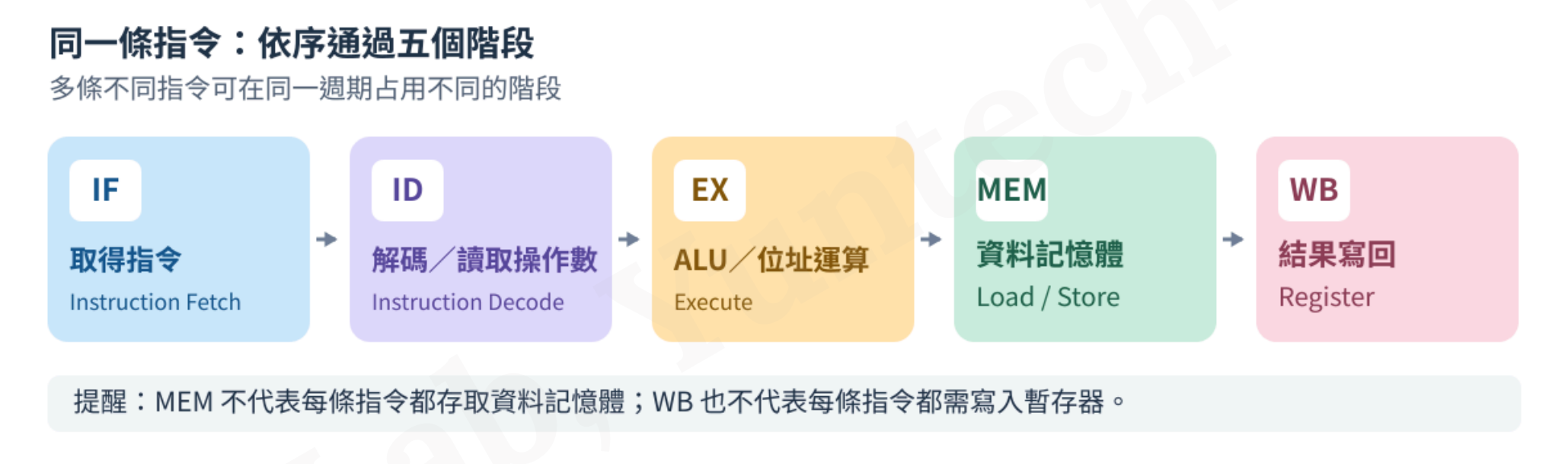
Arm 的五階段管線（Five-stage Pipeline）

五階段是理解經典 RISC / 部分 Arm 核心的常見教學模型：

1. **IF（Instruction Fetch，取指）**：取得指令。
2. **ID（Instruction Decode，解碼）**：解碼與準備操作數。
3. **EX（Execute，執行）**：ALU 運算或計算有效位址。
4. **MEM（Memory Access，記憶體存取）**：有需要才存取資料記憶體。
5. **WB（Write Back，寫回）**：視指令種類寫回運算結果。

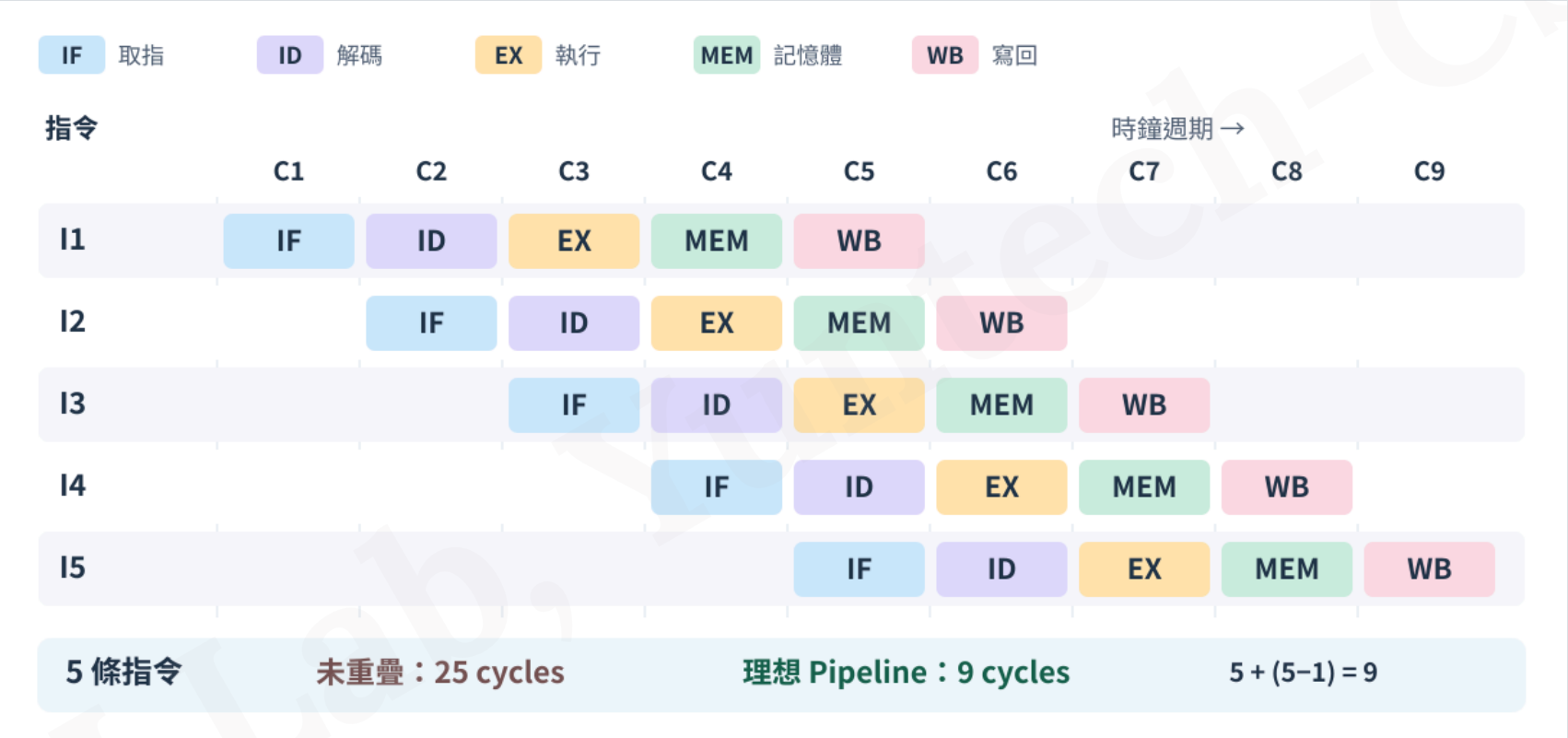
注意： 三階段 Execute 的工作被五階段模型進一步拆分；**不是**所有 Arm 核心都固定採五階段管線。

五階段 Pipeline：各階段如何分工？



閱讀重點： 一條指令依序通過 IF → ID → EX → MEM → WB；多條不同指令在同一個週期可以分別占用不同階段。MEM 不代表每條算術指令都會讀寫資料記憶體。

五階段管線執行範例：5 條指令 × 9 cycles

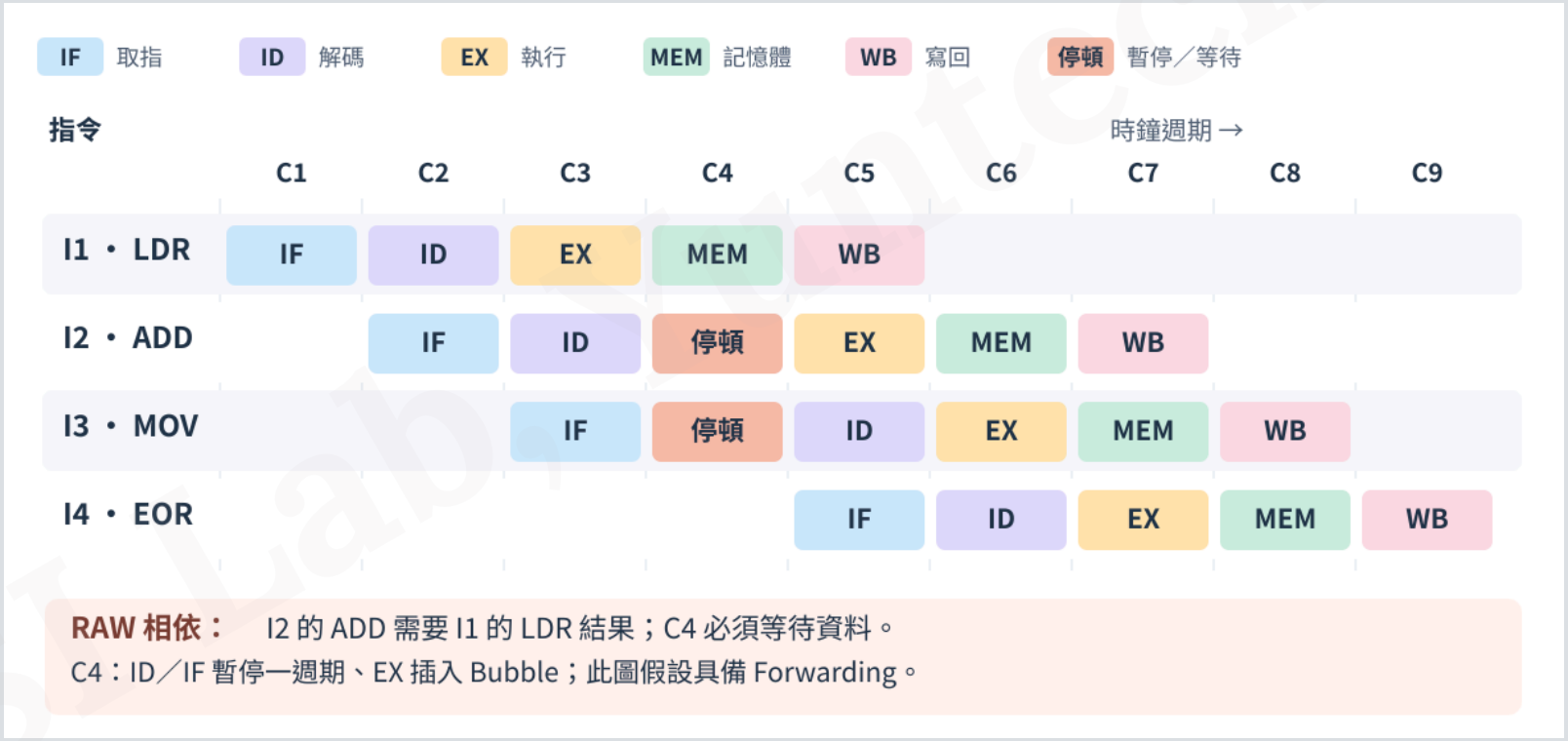


- 未重疊：5 條指令 × 5 cycles ＝ **25 cycles**。
- 理想五階段 Pipeline：5 + (5 − 1) ＝ **9 cycles**。

重點： **Pipeline 主要改善的是 Throughput（吞吐量）；單條指令在此模型中仍要經過五個階段，並非只花一個 cycle 就完成。**

為何 Pipeline 可能停頓？Load-Use Hazard

```
LDR R1, [R0]          @ I1：先從記憶體載入資料到 R1
ADD R2, R1, #1        @ I2：立刻需要 R1 的新值
```



重點： I2 要**讀取** I1 尚未產生完成的結果，形成 **RAW（Read After Write，讀後寫）資料相依**。在這張理想化五階段、具 Forwarding（資料轉送）的圖中仍需插入 **1 個 Bubble（停頓週期）**；真實情況依核心而異。

Arm CPU：更深的 Pipeline 設計

某些高效能 Arm 核心將處理工作切成更多階段，目的之一是縮短每階段的邏輯延遲，讓時脈有機會提高。

- **Cortex-A55**：強調能源效率，採 In-order（循序執行）。
- **Cortex-A78 / A710**：兼顧效能及能效，採 Out-of-order（亂序執行）。
- **Cortex-X / Neoverse 系列**：部分型號重視較高指令平行性與吞吐量。

不可混淆： **Pipeline 深度、Superscalar（超純量）及 Out-of-order 是不同的設計面向；新款 Arm 處理器不一定都使用很深的管線，確切階段數須查特定核心規格。**

深度 Pipeline：階段如何進一步細分？



重點： 圖中三、五、更多階段為概念對照，不代表圖上每一種分段都屬於某款特定 Arm CPU；深度增加不保證 IPC（Instructions Per Cycle，每週期完成指令數）必然上升。

多指令同時派發（Multiple-issue / Superscalar）

- **單發射（Single-issue）**：理想情況下，每個核心每週期處理一條新指令進入發射 / 執行流程。
- **多發射（Multiple-issue）**：有足夠硬體資源時，一個核心每週期可發射多條指令；例如 **2-wide** 表示相關發射階段最多兩條。
- **Superscalar（超純量）**：核心具備同週期處理多條指令的平行硬體能力；不等於單純增加 Pipeline 階段數。

重要： **Fetch、Decode、Dispatch、Issue、Retire 的「寬度」可能各不相同；多發射也不保證每個週期真的完成同樣多條指令。**

單核心與多核心：一個 Cycle 可以完成幾條？

Pipeline、Issue Width、Core Count：不是同一件事

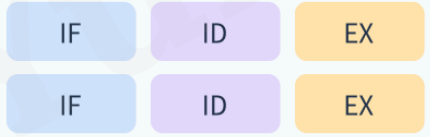
以下為理想化上限示意；假設各階段所需資源足夠、沒有 Stall，且四個核心的時脈相同。

單核心・1-wide
1 條／週期（理想）



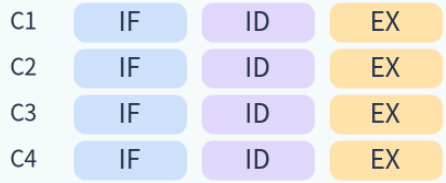
1 個核心 × 1 條

單核心・2-wide
2 條／週期（需雙路資源）



1 個核心 × 2 條

四核心・每核心 1-wide
全 CPU 理想 4 條／週期



4 個核心 × 各 1 條

注意：階段重疊 ≠ 同週期完成量；派發寬度也不必然等於實際 IPC。

- **Core Count（核心數）**：多個核心可同時執行彼此的指令流。
- **Issue Width（發射寬度）**：每個核心在特定管線階段，一個週期最多可處理幾條指令。
- **Pipeline Depth（管線深度）**：每條指令經過多少階段，並非每週期完成量。

重點： **一個核心的三階段管線，可同時有三條指令正在不同階段，卻通常只在理想情況每週期完成一條；若核心支援 2-wide，理論上才可能在足夠資源下每週期完成多條。**

IPSI Lab, Yuntech-CSIE

Pipeline 深度、發射寬度、核心數與亂序：四個不同問題

設計面向	核心問題	不能誤解為
Pipeline Depth	一條指令被切成幾個階段？	有 5 階段就能完成 5 條 / cycle
Issue Width	一個核心每週期最多發射幾條？	2-wide 一定能完成 2 條 / cycle
Core Count	有多少核心能各自執行工作？	4 cores 讓一條依序程式快 4 倍
Out-of-order（OoO）	就緒的獨立指令能否先執行？	必須採深 Pipeline 才有 OoO

簡化估算： 若各核心時脈與 IPC 相近、工作可充分平行，整體完成指令率約為 `時脈頻率 × 每核心 IPC × 核心數`；實際還受相依性、記憶體與工作負載限制。

為什麼部分高效能 Arm 核心採用較深 Pipeline？

- **好處：** 將較長的組合邏輯拆成更短的階段，有機會提高可用的 Clock Frequency（時脈頻率）。
- **代價：** 更多管線暫存器、控制及功耗開銷；單條指令的週期延遲與相依等待可能增加。
- **Data Hazard（資料危障）**：來自指令相依，須以 Forwarding、Scheduling、Stall 等機制處理；**深管線本身不會自動消除 Hazard**。
- **Out-of-order 與 Multiple-issue**：可改善部分指令平行處理，但不是「Pipeline 越深」自然獲得的能力。

結論： **深度、寬度、時脈與實際 IPC 必須一併評估，不存在「管線越深就一定越快」的定律。**

分支預測（Branch Prediction）

當 `B.EQ` 等條件分支的結果尚未確定時，CPU 若停下取指就可能產生空轉。

- **Branch Prediction（分支預測）**：先預測下一個要取指的方向 / 目標。
- **預測正確**：減少等待，讓 Pipeline 能持續處理更多指令。
- **預測錯誤**：捨棄錯誤路徑上的部分工作，改從正確位置重新取指。

與前面條件執行單元銜接： A64 的 `B.cond` 使用條件分支；`CSEL`、`CCMP` 則可表達某些不必以分支實作的條件邏輯。

深 Pipeline 的成本：Branch Misprediction（分支預測錯誤）

- 分支預測錯誤時，CPU 必須 **Flush（清除）**錯誤路徑上的尚未提交工作，重新載入正確路徑的指令。
- 深 Pipeline **可能**使錯誤路徑上累積更多工作、增加恢復成本；但代價仍取決於分支解析位置與特定微架構。
- 不能一律宣稱「必定損失 20 cycles」或「預測準確率固定超過 95%」。

本小節總結： **Pipeline 讓不同指令的階段重疊；Clock Cycle 決定同步節奏；Issue Width 決定單核心的平行發射能力；Core Count 決定可並行運作的核心數。理解這四者後，才能正確討論時脈、吞吐量、Hazard 與分支預測。**

Armv7 與 Armv8 架構比較

Armv7 與 Armv8 的架構特點（1）：執行狀態與指令集

比較範圍： 以 Armv7-A 的 AArch32 與 Armv8-A 的 AArch64 為主。Armv8-A 也定義 AArch32，但是否可用取決於核心實作。

比較項目	Armv7-A（AArch32）	Armv8-A（以 AArch64 為主）
執行狀態	AArch32：32-bit 執行模型	AArch64：64-bit 執行模型；部分核心另支援 AArch32
指令集	A32：固定 32-bit 編碼；T32：16 / 32-bit 編碼	AArch64 使用 **A64**（固定 32-bit 編碼）；若支援 AArch32，可使用 A32 / T32
整數暫存器	R0–R12：13 個一般用途；R13=SP、R14=LR、R15=PC	X0–X30：31 個 64-bit 暫存器；W0–W30 為其 32-bit 視圖；另有 SP，PC 非一般用途暫存器

閱讀重點： 「64-bit 執行狀態」不代表「每條指令長 64 bits」；A64 指令仍使用固定 32-bit 編碼。

Armv7 與 Armv8 的架構特點（2）：旗標、特權與呼叫慣例

比較項目	Armv7-A（AArch32）	Armv8-A（以 AArch64 為主）
條件旗標 / 條件執行	CPSR.NZCV；A32 多數指令可加條件後綴，例如 `ADDEQ`	PSTATE.NZCV；A64 使用 `B.cond`、`CSEL`、`CCMP` 等專用指令
特權 / 例外處理	User、Supervisor、IRQ、FIQ 等處理器模式；擴充包含 Hyp / Monitor	AArch64 使用 EL0–EL3 例外層級（視實作）；AArch32 仍有處理器模式
函式呼叫慣例	AAPCS32：一般整數參數常由 R0–R3 傳遞	AAPCS64：一般整數參數常由 X0–X7 傳遞

IRQ = Interrupt Request；FIQ = Fast Interrupt Request；EL = Exception Level；AAPCS = Procedure Call Standard for the Arm Architecture。

關鍵區分：AArch32 / AArch64 是執行狀態；User / Supervisor / IRQ 或 EL0–EL3 是特權與例外處理概念，不能當成同一種分類。

管線設計與超純量：四個具體核心的比較

同一架構世代，也可能有不同的 Pipeline 深度與執行方式：

架構版本	處理器核心	Pipeline（代表性資料）	指令執行方式
Armv7-A	**Cortex-A7**	**8 階段**	**In-order**；部分指令可雙發射
Armv7-A	**Cortex-A15**	**15 階段以上**	**Out-of-order**；超純量設計
Armv8-A	**Cortex-A53**	**8 階段**	**In-order**；可有限度雙發射
Armv8-A	**Cortex-A72**	**15 階段以上**	**Out-of-order**；超純量設計

觀察： **Armv7-A 與 Armv8-A 都有不同深度、不同指令執行方式的核心；不能僅由架構版本推定 Pipeline 與效能。**

IPSI Lab, Yuntech-CSIE

管線比較的結論：架構與微架構不能混為一談

微架構特性	回答的問題
Pipeline Depth（管線深度）	一條指令的處理被細分為多少個階段？
Superscalar（超純量 / 多發射）	單一核心在一個週期內，能否發射多條指令？
Out-of-order（亂序執行）	是否能讓準備就緒的指令先執行，而不完全依原始順序等待？

- **In-order 不代表單發射。** Cortex-A7 是循序執行核心，仍能對部分指令雙發射。
- **產品型號不等於架構版本。** ARM7TDMI（三階段，Armv4T）及 ARM926EJ-S（五階段，Armv5TE）都早於 Armv7-A；不能將「ARM7 / ARM9」直接當成「Armv7 / Armv9」。

總結： **Armv7-A / Armv8-A 指的是架構；Pipeline 深度、多發射及亂序執行，則須查看具體 CPU 核心。**

補充：釐清 Arm 產品的層級

在比較 Armv7 和 Armv8 之前，必須先釐清「架構」、「核心」、「晶片」的區別：

- **Arm 架構 (Architecture)**：
  - 定義指令集（ISA）和暫存器規範的「規格書」。
  - 範例： **Armv7**, **Armv8**。
- **Arm 核心 (Core)**：
  - Arm 公司依照「架構規範」設計的「處理器 IP（Intellectual Property，矽智財)」。
  - 範例： **Cortex-A**, **Cortex-R**, **Cortex-M** 系列。
- **晶片 (SoC - System on Chip)**：
  - 晶片設計者可能授權 Arm 核心 IP，或取得 Arm 架構授權自行設計 CPU；再整合記憶體控制器、GPU（Graphics Processing Unit，圖形處理器） / NPU（Neural Processing Unit，神經網路處理器）等元件形成 SoC。
  - 範例： Apple M1, 高通 Snapdragon 8 Gen 2。

Arm 核心 (Core)：Cortex 核心產品線

- **Cortex-A (Application)**：
  - 架構例：Armv7-A、Armv8-A 等。
  - 用途：行動裝置、通用運算；**可**執行 Linux、Android 等作業系統，但非架構規定必須使用。
- **Cortex-R (Real-time)**：
  - 架構例：Armv7-R、Armv8-R。
  - 用途：需即時性與可預測延遲的控制系統。
- **Cortex-M (Microcontroller)**：
  - 架構例：Armv6-M、Armv7-M、Armv8-M；實際支援的 T32 指令子集依核心而異。
  - 用途：低功耗、低成本的微控制器。

Armv7 架構

暫存器架構

- **通用暫存器（R0 - R12）**：用於一般資料處理。
- **特殊暫存器**：
  - **R13（SP）**：堆疊指標
  - **R14（LR）**：連結暫存器
  - **R15（PC）**：程式計數器 (可被指令直接讀寫)
- **CPSR**：包含條件碼，用於記錄執行狀態。

IPSI Lab, Yuntech-CSIE

指令集

- **A32 指令集**（ARM）：提供標準 32 位元指令集。
- **T32 指令集**（含 Thumb-2 擴充）：
  - 依架構與核心支援 16-bit 及 32-bit 編碼形式，可提高程式碼密度。
  - **注意：ARM926EJ-S（Armv5TE）可使用其世代的 Thumb，但不具備 Armv7 的完整 Thumb-2 指令功能**；不能只由指令長度推斷執行效能。

管線設計

- **三階段管線**：早期 ARM7 核心曾使用，是微架構範例，並非所有 Armv7 CPU 的共同設計。
- **五階段管線**：可作為教學上的經典 RISC 模型；ARM9（較早架構）等核心有不同管線實作，不能由 Armv7 版本推定階段數。

Armv8 架構

暫存器架構（AArch64）

- **X0 - X30**：31 個 64 位元通用暫存器。
  - W0 - W30 則是 32 位元版本
- **X30（LR）**：`LR` 是 **X30 的別名**，常用於儲存函式返回位址。
- **SP**：專用的堆疊指標 (Stack Pointer)。
- **XZR (WZR)**：零暫存器 (Zero Register)，讀取恆為 0。
- **PC (Program Counter)**：
  - **重大變革：** PC **不再是**通用暫存器 (GPR)，無法被 `MOV` 等指令直接存取。
- **NZCV 條件旗標**：屬於 **PSTATE** 的一部分，並可透過相關系統暫存器存取。

IPSI Lab, Yuntech-CSIE

指令集

- **A64 指令集**：AArch64 使用的指令集，支援 64-bit 整數運算，**每條指令仍是 32-bit 編碼**；效能優劣視微架構與工作負載而定。
- **執行狀態**：
  - **AArch64**：執行 A64 指令集。
  - **AArch32**：執行 A32/T32 指令集；**實際支援取決於核心型號與執行層級**。
- **架構變革**：
  - A64 **不採用 A32 那種普遍附加條件後綴**的設計，而使用 `B.cond`、`CSEL` 等指令；不是完全取消條件執行。

管線設計

- **管線深度由微架構決定**：
  - Armv8-A 並未規定「六階段」或其他固定階段數；不同核心可採不同深度。
- **超純量 (Superscalar)**：
  - 部分高效能核心能在同一週期處理多條指令，但**並非 Armv8 必備**。
- **亂序執行 (Out-of-Order, OoO)**：
  - 部分 Cortex-A/Neoverse 核心可讓已就緒指令先執行，降低相依或記憶體等待造成的停頓；**也有 in-order 核心**。

IPSI Lab, Yuntech-CSIE

從 Arm 架構知識銜接到組合語言與工具鏈

本單元先學會辨認：

- **AArch32**：R0–R15、LDR/STR、BL/BX、AAPCS32；後續組合語言範例採此脈絡。
- **AArch64**：X0–X30、A64、CSEL/CCMP；用於架構比較，**不是後續 VersatilePB 範例的指令集**。
- **Pipeline**：屬於處理器微架構；QEMU/GDB 的指令單步通常觀察的是暫存器、記憶體等**架構狀態**，不是每一級硬體管線。

接著在 CPU-02：把 AArch32 指令寫成 `.s`，辨識資料段、程式段與函式介面。

再接 Toolchain：Windows / MSYS2 → `arm-none-eabi` → QEMU `versatilepb`（預設 ARM926EJ-S，Armv5TE）→ GDB。

架構比較與實驗平台分開處理；`syscalls.c` 暫沿用既有可運行版本。